

汇编语言大作业 4 实验报告

小组成员：吴竞宸、黄奕皓、张晨轩、张诗炫

2026/06/20

1 概述

本次实验要求实现 4096×4096 整数矩阵乘法，并采用多种优化方法加速。我们实现了五种算法：

1. Python 朴素矩阵乘法（作为性能参考基准）
2. C 语言朴素矩阵乘法（ikj 循环顺序 + 稀疏跳过）
3. 分块矩阵乘法（ $BS = 64$ ，汇编核心）
4. SIMD AVX2 向量化矩阵乘法（B 矩阵转置 + AVX2 点积汇编）
5. OpenMP 多线程 + SIMD AVX2（四线程并行，B 转置后调用 AVX2 汇编核心）

所有 C 版本的核心计算函数均采用 x86-64 AT&T 语法手写汇编实现。项目沿用汇编大作业 2 的二进制数据流水线：`generate.c` 生成测试数据 → 计算程序读写 `.bin` 文件 → `check.c` 统一验证正确性。项目根目录提供 Makefile 一站式编译运行。

2 实验环境

- 宿主机：Windows 11，CPU Intel Core i9-14900HX (2.20 GHz)
- 虚拟机：VMware Workstation，Ubuntu 24.04.3，分配 4 核 6 GB 内存
- Python 环境：Windows 宿主 Python 3.x（仅 Python 版本在宿主机运行）
- 编译器：GCC 11.4.0，编译选项 `-O2 -m64`（SIMD 版附加 `-mavx2`，OpenMP 版附加 `-fopenmp`）
- 计时方式：`clock_gettime(CLOCK_MONOTONIC)` 获取实际墙钟时间
- 矩阵规模： 4096×4096 ，元素为 `int` 类型（0-9 随机整数），每个矩阵约 64 MB

3 实验结果

3.1 性能数据

算法	运行时间 (s)	加速比
Python 朴素	3541.89	—
C 朴素	29.50	1.00
C 分块	33.89	0.87
C SIMD AVX2	15.77	1.87
C OpenMP + SIMD AVX2	4.50	6.56

表 1: 各算法运行时间与加速比

3.2 正确性验证

所有版本的输出均通过 `check.c` 验证，验证方式为：与 Python 参考实现的逐个元素比较（左上角 8×8 全检 + 随机 200 个位置抽样），全部通过。

4 算法分析

4.1 Python 朴素版本

Python 版本使用三重循环 `i-k-j` 顺序，并在 $A[i][k] = 0$ 时跳过内层循环以利用测试数据的稀疏性（0-9 随机整数中约 10% 为零）。但解释型语言的巨大开销使得 4096×4096 矩阵乘法耗时近一小时，仅作为正确性验证的基准参考，不具备实用价值。

4.2 C 朴素版本

C 朴素版本采用 `ikj` 循环顺序而非传统的 `ijk`。这种顺序的优点是：最内层循环访问 $B[k][j]$ 时 k 固定不变， j 连续递增，而 $B[k]$ 是一整行连续存放的数据，因此每次访存几乎必定命中 L1 缓存。同时，`aik` 在外层被提出，内层只做一次乘法和一次加法，减少了内存访问次数。 $A[i][k] = 0$ 时的稀疏跳过进一步减少了约 10% 的无效计算。最终耗时 29.50 s。

4.3 分块矩阵乘法（汇编核心）

分块乘法将 4096×4096 矩阵按 64×64 划分，外层 `ii-jj-kk` 循环遍历所有块，内层由汇编函数 `block_kernel` 处理单个块。汇编核心采用 `k-j` 顺序：对每个 k ，加载一行 $B[k][j_0 \dots j_0 + 63]$ ，计算结果连续累加到 $C[i][j]$ 中。由于块足够小， $B[k]$ 的 64 个元素可以完全放在 L1 缓存中复用。

关键指令	说明
<code>imulq \$(N*4), %rax</code>	计算行首偏移 $i \times 4096 \times 4$ 字节
<code>lea (%rsi,%rax,1), %rax</code>	计算 $B[k]$ 行首地址
<code>mov (%rax,%rbx,1), %ecx</code>	加载 $B[k][j]$ (字节偏移寻址, 消除乘法)
<code>imull %r8d, %ecx</code>	标量乘法 $A[i][k] \times B[k][j]$
<code>addl %ecx, (%r14,%rbx,1)</code>	累加到 $C[i][j]$

表 2: block_kernel.s 关键指令

然而分块版本耗时 33.89 s, 比 C 朴素版 (29.50 s) 更慢, 加速比仅为 0.87。分析原因:

- (1) 汇编核心中 $B[k]$ 行首地址的计算使用了 `imulq` (64 位乘法), 且每次循环都要重新计算;
- (2) 内层 j 循环使用字节偏移 `add $4, %ebx` 配合基址 + 索引寻址, 虽然避免了 $j \times 4$ 的乘法, 但每迭代只处理一个元素, 未充分利用指令级并行;
- (3) 分块本身产生的额外外层循环开销 ($64^3 = 262144$ 次 `block_kernel` 调用) 抵消了缓存局部性的收益。

4.4 SIMD AVX2 向量化 (汇编核心)

该版本先对矩阵 B 做一次转置得到 BT , 此后对每个元素对 (i, j) , 计算 A 的行 $A[i][*]$ 与 BT 的行 $BT[j][*]$ (即原 B 的列) 的点积。由于两行都连续存放, 访存模式完全顺序, 缓存利用率最优。

汇编函数 `simd_dot` 一次处理 8 个 32 位整数。先将 B 转置为 BT (C 语言循环实现), 然后 C 端双重循环遍历所有 (i, j) , 每个 (i, j) 调用 `simd_dot` 计算完整点积。

关键指令	说明
<code>vmovdqu (%rdi,%rax,4), %ymm1</code>	从 $A[i]$ 加载 8 个 int (256 位) 到 ymm 寄存器
<code>vmovdqu (%rsi,%rax,4), %ymm2</code>	从 $BT[j]$ 加载 8 个 int 到 ymm 寄存器
<code>vpmulld %ymm2, %ymm1, %ymm3</code>	8 对 int 并行乘法 (单条指令完成 8 次乘法)
<code>vpadd %ymm3, %ymm0, %ymm0</code>	8 对 int 并行加法, 累加到向量累加器
<code>vextracti128 \$1, %ymm0, %xmm1</code>	将 256 位 ymm 的高 128 位提取到 xmm
<code>vphadd %xmm0, %xmm0, %xmm0</code>	水平成对加法, 规约 8 个 lane 到 1 个标量

表 3: simd_dot.s 关键 AVX2 指令

该版本耗时 15.77 s, 加速比 1.87。AVX2 单条指令替代 8 次标量乘法/加法, 理论上可获得 8 倍加速。但实际加速比受到三个因素制约: B 矩阵转置的开销 (遍历 4096^2 个元素并写回)、调用 `simd_dot` 时 4096^2 次函数调用开销、以及水平归约 (`vextracti128 + vpsltd + vphadd`) 需要多条指令完成。

4.5 OpenMP + SIMD AVX2

在 SIMD AVX2 的基础上，将外层 i 循环用 `#pragma omp parallel for` 并行化。B 矩阵事先统一转置（串行，仅执行一次），随后四个线程各负责一部分行的计算，每个线程内部仍调用 AVX2 汇编核心 `simd_dot`。

该版本耗时 4.50 s，加速比达到 6.56。这一结果同时体现了线程级并行（利用多核 CPU）和指令级并行（AVX2 单指令多数据）的收益。OpenMP 调度策略为 `schedule(static)`，确保各线程负载均衡。

5 总结

本次实验从 Python 朴素实现出发，逐步引入 ikj 循环重排、分块缓存优化、AVX2 向量指令、以及 OpenMP 多线程并行，将 4096×4096 整数矩阵乘法的运行时间从约 59 分钟（Python）压缩至 4.5 秒（OpenMP + SIMD），总体加速约 787 倍。汇编核心函数的编写使我们深入理解了 x86-64 指令集、SIMD 向量化编程和缓存优化技术。